PRINCIPAL COMPONENT ANALYSIS (PCA) AND ITS APPLICATION IN FACIAL RECOGNITION

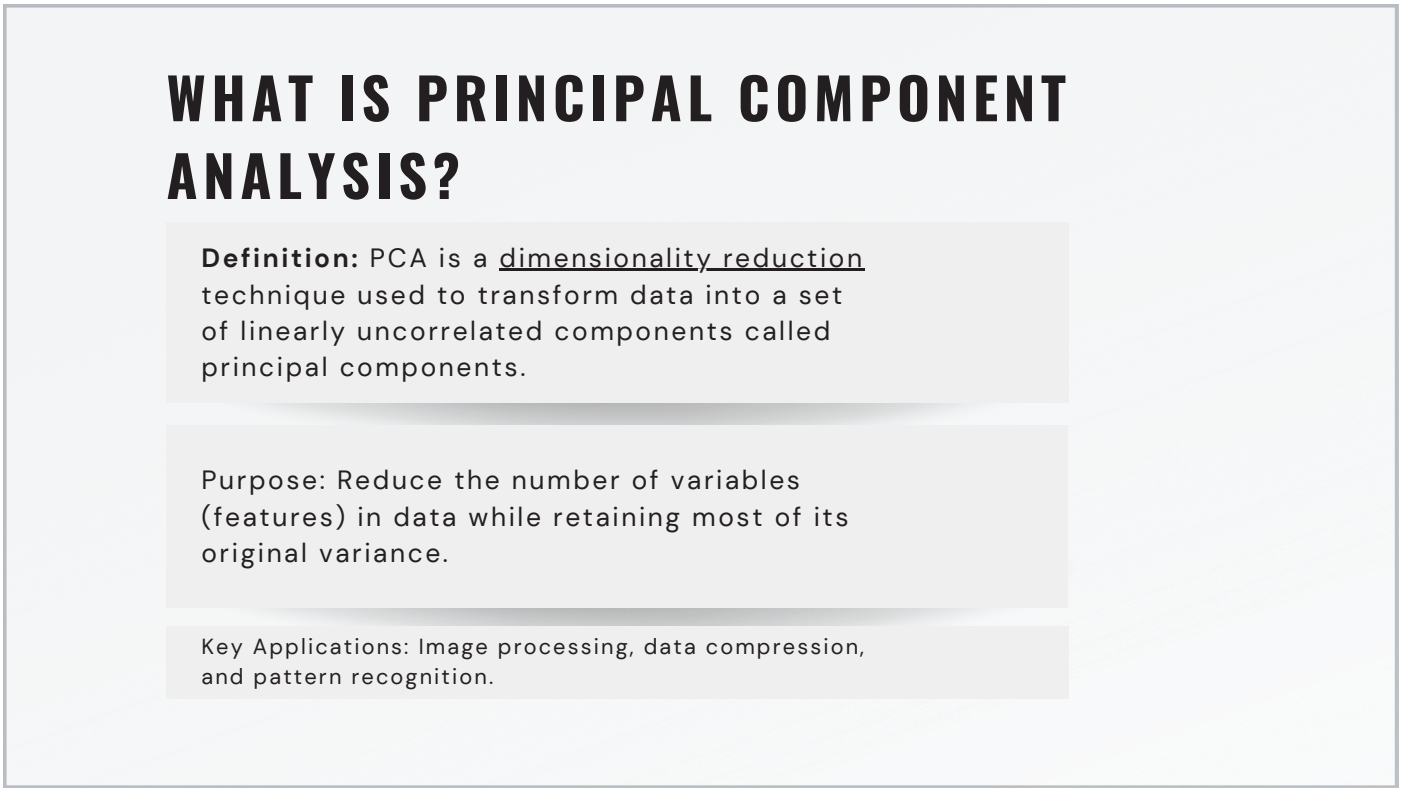
EXPLORING THE MATHEMATICS AND APPLICATION IN COMPUTER VISION

OUR TEAM

Hon Hao Yuan	Darwin Alexander	Andrew Tanujaya
洪浩元	许固杨	谭加亚
1820222026	1820212017	1820222027

CONTENT

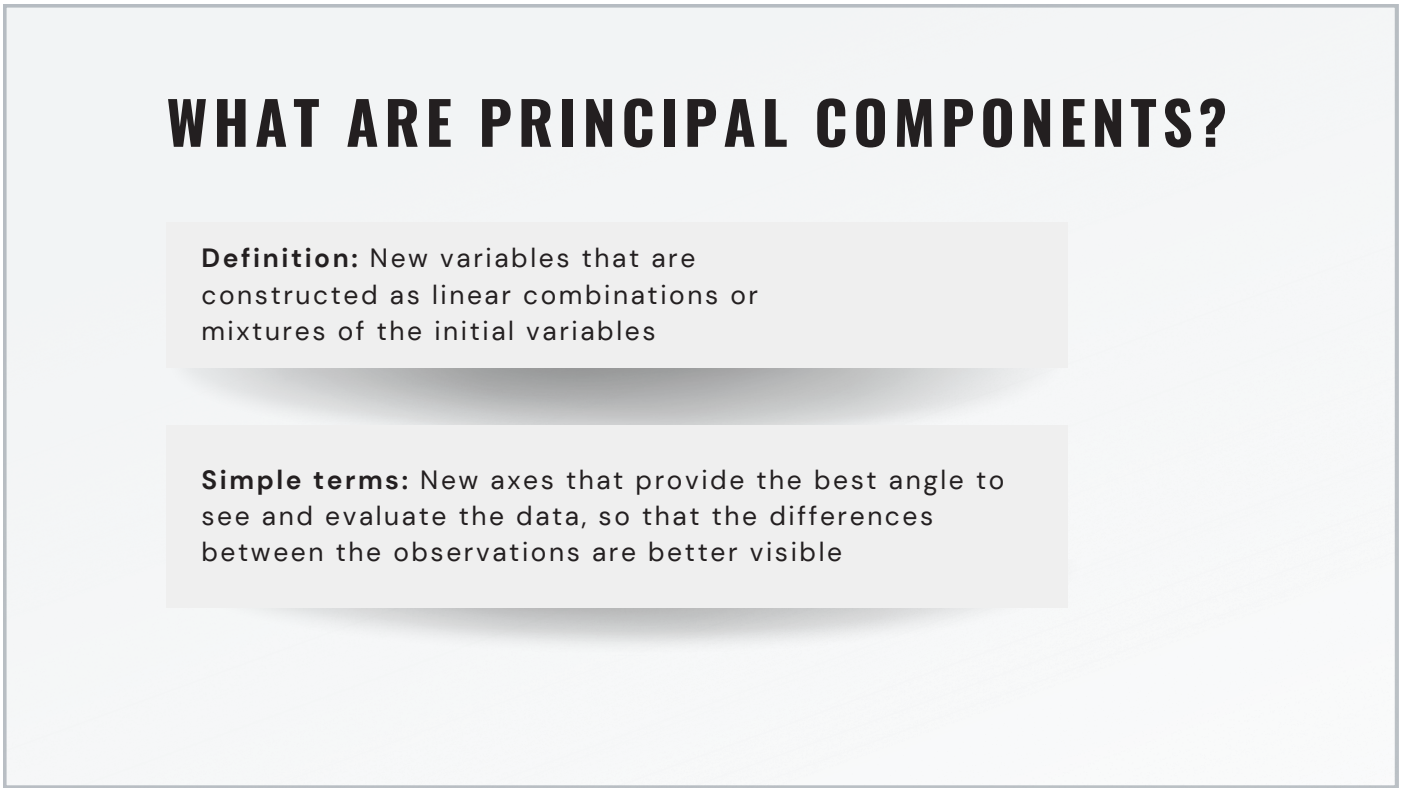
INTRODUCTION TO PCA



Principal component analysis, or PCA, is a dimensionality reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.

However, reducing the number of variables of a data set naturally comes at the expense of accuracy, but the trick in dimensionality reduction is to trade a little accuracy for simplicity. Because smaller data sets are easier to explore and visualize, and thus make analyzing data points much easier and faster for machine learning algorithms without extraneous variables to process.

So, to sum up, the idea of PCA is simple: reduce the number of variables of a data set, while preserving as much information as possible.



Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables. These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components. So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shown in the scree plot in the next slide.

So, here's the graph. For a quick example, imagine you're shopping for fruit. You're keeping track of details like color, sweetness, size, weight, price, and shelf life—six characteristics for each fruit. Since each characteristic is like an initial variable, you're dealing with 6-dimensional data.

With hundreds of fruits, remembering all these details gets overwhelming. This is where PCA helps by reducing the number of variables you need while keeping essential information.

Principal Components are new variables created by combining the original characteristics to capture key patterns across the fruits.

The first principal component finds the most important pattern, such as a mix of sweetness, size, and price that varies the most.
The second component captures the next pattern, like shelf life and weight, and is uncorrelated with the first.
PCA continues this way, but typically, only the first two or three components are needed to understand differences between the fruits. So, instead of tracking six characteristics, you focus on just two or three to simplify without losing important information.

UNDERSTANDING PCA COMPUTATION

STEP 1: STANDARDIZATION

- Equalizes the scale of all variables
- Ensures no single variable dominates due to range differences
- Formula:

$$z = \frac{value - mean}{standard\ deviation}$$

To start with PCA computation, we first standardize the data. The goal here is to make sure each variable contributes equally, so no variable overpowers the others due to differences in scale. For example, if one variable ranges from 0 to 100 while another ranges from 0 to 1, the first would have a much larger impact on the results.

To prevent this bias, we transform each variable to the same scale by subtracting its mean and dividing by its standard deviation, as shown here. This step is essential for a fair and balanced analysis.

STEP 2: COVARIANCE MATRIX COMPUTATION

- Reveals relationships between variables
- Captures redundant information through correlations
- Diagonal = Variances; Symmetric entries show covariances

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

In the second step, we calculate the covariance matrix to capture relationships between variables. This matrix helps us see if any variables are strongly correlated, meaning they may contain overlapping information.

Let's look at this example: it's a 3x3 covariance matrix for three variables, say, x, y, and z. Each diagonal entry shows the variance of a single variable, while the off-diagonal entries show covariances between pairs, like x with y, or y with z. The symmetry of the matrix means covariances are mirrored above and below the diagonal.

Positive covariances indicate that variables tend to increase or decrease together, while negative covariances mean one goes up as the other goes down. This table of relationships helps us prepare for the next step in PCA.

STEP 3: COMPUTE EIGENVECTORS AND EIGENVALUES

- Identifies principal components
- Eigenvectors = directions of maximum variance
- Eigenvalues = amount of variance in each component

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \qquad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \qquad \lambda_2 = 0.04908323$$

In this 3rd step, we compute the eigenvectors and eigenvalues of the covariance matrix to find our principal components. Eigenvectors and eigenvalues work together—each eigenvector points in a direction where there is maximum variance in the data, and the corresponding eigenvalue tells us how much variance exists in that direction.

In the example shown, we have two eigenvectors, v1 and v2, each with an eigenvalue. By ranking these eigenvectors by their eigenvalues from highest to lowest, we identify the principal components in order of importance. Here,
v1 is the first principal component, as it has the highest eigenvalue, meaning it captures the most variance.

By dividing each eigenvalue by the total sum of eigenvalues, we can see that the first component, v1, captures 96% of the data's variance, while the second component, v2, captures only 4%. This tells us that most of the information is concentrated in the first component.

STEP 4: CREATE A FEATURE VECTOR

- Choose components to keep based on eigenvalues
- Form a matrix with selected eigenvectors (Feature Vector)
- Discarding low-variance components reduces dimensionality

$$\begin{bmatrix} 0.6778736 & -0.7351785 \\ 0.7351785 & 0.6778736 \end{bmatrix} \xrightarrow{\text{Discard v2}} \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix}$$

In Step 4, we create the Feature Vector, which is a matrix containing the principal components we choose to keep. We first rank the components by their eigenvalues in descending order, then decide whether to keep all or only the most significant ones.

In our example, we can create a Feature Vector with both eigenvectors, v1 and v2, or discard v2 , which has lower significance, keeping only v1. If we keep only v1, we reduce the dimensionality of the data from 2 to 1, with only a small loss of information—just 4%, as v1 captures 96% of the data's variance.

This choice depends on your goal: if you need to reduce dimensions while retaining most information, discarding low-variance components can be helpful. However, if dimensionality reduction isn't needed, you can keep all components for a complete data representation.

STEP 5: RECAST DATA ALONG PRINCIPAL COMPONENT AXES

- Reorient data using principal components
- Multiply data by the feature vector to transform into new axes

$$FinalDataSet = FeatureVector^T * StandardizedOriginalDataSet^T$$

In the final step, we recast the data along the axes of the principal components. So far, we've identified the principal components and created the feature vector, but our data still uses the original variables as its axes.

To complete the transformation, we multiply the transpose of our original data by the transpose of the feature vector. This reorients the data onto the principal component axes, where each component represents a combination of the original variables.

This transformation allows us to view the data in terms of its most significant patterns, simplifying analysis and highlighting the main structures in our data.

FACE RECOGNITION WITH PCA

PREPROCESSING DATA INPUT

- Preprocessing for PCA: Essential for effective performance and meaningful insights in PCA.
- Sensitivity to Scale: PCA relies on variance, so features with larger ranges may dominate.
- Standardization: First step to balance features by giving each a mean of 0 and standard deviation of 1.
- Purpose of Standardization: Ensures PCA evaluates features equally, focusing on data structure.

Firstly, preprocessing data is a crucial step before applying (PCA) in machine learning, as it helps PCA perform effectively and yield meaningful insights. So firstly we preprocess PCA to be sensitive to scale and giving it its standardization.

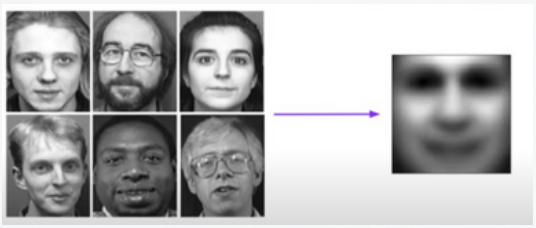
As PCA is sensitive to the scale of data because it calculates variance to find principal components.

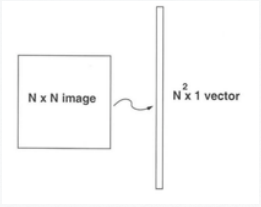
Standardization allows PCA to evaluate each feature equally, focusing purely on the structure of the data.

So, features with larger numerical ranges can dominate others, skewing the results. Therefore, standardizing data—ensuring each feature has a mean of 0 and a standard deviation of 1—is often the first step.

STEP 1 : GREYSCALE AND MEAN SUBSTRACTION

- Greyscale: Convert facial images from color (RGB) to greyscale, to reduce complexity of data.
- Mean Subtraction: Calculate the mean image from the training dataset and subtract this mean from each image, change image matrix to vectors





$$\psi = \frac{1}{m}\sum_{i=1}^{m} x_i$$

$$a_i = x_i - \psi$$

calculates the average of all these face vectors and subtract it from each vector

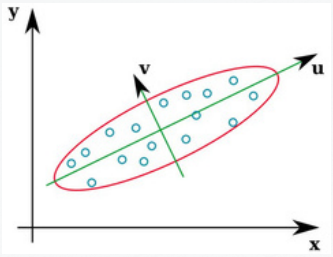
Greyscale:
Convert facial images from color (RGB) to greyscale. This reduces the complexity of the data without losing essential features, as color information is often not critical for facial recognition tasks.

Mean Subtraction:
Calculate the mean image from the training dataset and subtract this mean from each image. This centers the data around zero, which is important for PCA as it helps in identifying the directions of maximum variance.

So in here, the goal is to calculate the average of all these face vectors and substract them from each other. Which the example we can see on the left image, regular images is substracted by the average of those images to generate a vague image all the images combined.

STEP 2 : COVARIANCE MATRIX EIGENVECTORS AND EIGENVALUES

- Covariance Matrix: The covariance matrix captures how pixel values vary together across the dataset.
- Eigenvectors: Each eigenvector corresponds to a direction in the image space.
- Eigenvalues: Indicate the amount of variance captured by their corresponding eigenvectors.



$$A^T A\nu_i = \lambda_i \nu_i$$

$$AA^T A\nu_i = \lambda_i A\nu_i$$

$$x_i - \psi = \sum_{j=1}^{K} w_j u_j$$

$$A = [a_1 \quad a_2 \quad a_3 \quad .... \quad a_m]$$

take all face vectors so that we get a matrix of size of N2 * M

$$Cov = A^T A$$

$$C' u_i = \lambda_i u_i$$

calculate eigen values and eigenvectors of above covariance matrix

Covariance Matrix:
Construct the covariance matrix from the mean-subtracted images. The covariance matrix captures how pixel values vary together across the dataset.

Eigenvectors and Eigenvalues:
By Compute the eigenvectors and eigenvalues of the covariance matrix. Each eigenvector corresponds to a direction in the image space, and the eigenvalues indicate the amount of variance captured by their corresponding eigenvectors.

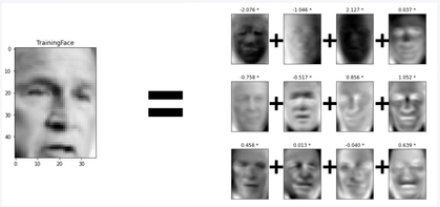
Here the goal is to find the eigenvectors and eigenvalues for further processing later on for the eigenfaces.

Second Formula
Now, we find Covariance matrix by multiplying A with AT. A has dimensions N2 * M, thus AT has dimensions M * N2. When we multiplied this gives us matrix of N2 * N2, which gives us N2 eigenvectors of N2 size which is not computationally efficient to calculate. So we calculate our covariance matrix by multiplying AT and A. This gives us M * M matrix which has M (assuming M << N2) eigenvectors of size M. Making the calculation more efficient

Fourth Formula
We take the normalized training faces (face – average face) x_{i}and represent each face vectors in the linear of combination of the best K eigenvectors

STEP 3 : CALCULATE EIGENFACES OF TRAINING SET

- The eigenvectors associated with the largest eigenvalues are selected as "eigenfaces."
- Eigenfaces: Represent the most important features of the face images and can be visualized as weighted combinations of the original images.



$$x_i = \begin{bmatrix} w_1^i \\ w_2^i \\ w_3^i \\ \cdot \\ \cdot \\ w_k^i \end{bmatrix}$$

$$\phi = y - \psi$$

Eigenfaces:
The eigenvectors associated with the largest eigenvalues are selected as "eigenfaces." These eigenfaces represent the most important features of the face images and can be visualized as weighted combinations of the original images.

In this step, we take the coefficient of eigenfaces and represent the training faces in the form of a vector of those coefficients. Don't mind the formula, it just has different letters, y is the image's initial eigenvector coefficient and v stripe is the average eigenvectors.

The basic goal here is to seek a neutral face where these features can be found in each face image

STEP 4 : SELECT N BEST PRINCIPLE COMPONENT AND CALCULATE WEIGTHS FROM DATASET

- Selecting Components: Choose the top n eigenfaces (principal components) based on their eigenvalues, which capture the most variance in the data.
- Calculating Weights: For each training image, project it onto the selected eigenfaces to calculate a weight vector.

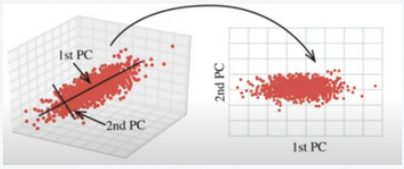
Selecting Components:
Choose the top n eigenfaces (principal components) based on their eigenvalues, which capture the most variance in the data. The choice of n is crucial as it balances between maintaining information and reducing dimensionality.

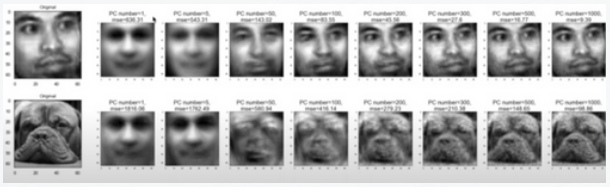
Calculate Weights:
For each training image, it project it onto the selected eigenfaces to calculate a weight vector. This vector represents the image in the reduced eigenspace formed by the eigenfaces.

What is basically means is that we want to find the best coefficient from the dataset, closest to the average eigenvector mentioned previously. This is done so that it can be set as an example for further AI face recognition engines.

STEP 5 : PROJECT UNKNOWN IMAGE TO EIGENFACES SPACE

- Projection: For an unknown image, perform the same preprocessing steps (greyscale conversion and mean subtraction) and then project this image onto the selected eigenfaces.





$$\phi = \sum_{i=1}^{k} w_i u_i$$

$$\Omega = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ . \\ . \\ w_k \end{bmatrix}$$

$$e_r = min_l \, \|\Omega - \Omega_l\|$$

Projection:
For an unknown image, perform the same preprocessing steps (greyscale conversion and mean subtraction) and then project this image onto the selected eigenfaces. This generates a weight vector that represents the unknown image in the same eigenspace.

Image on right side formula explains how to project the normalized vector into eigenspace to obtain the linear combination of eigenfaces.

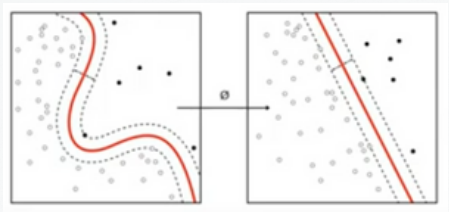
\phi = \sum_{i=1}^{k}w_{i}u_{i}
(formula bottom)

And then we take the vector generated in the above step and subtract it from the training image to get the minimum distance between the training vectors and testing vectors

This way, we can see if the face recognition image is the same face or not

STEP 6 : COMPARING ERROR WITH THRESHOLD VALUE

- Calculate the distance between the weight vector of the unknown image and those of the training images, then determine if the distance is below a set threshold to classify it as a match or unknown.



Has 2 steps

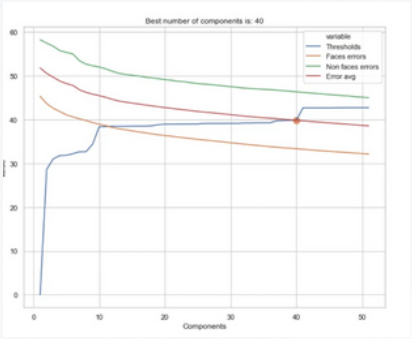
Error Calculation: Calculate the distance (e.g., Euclidean distance) between the weight vector of the unknown image and the weight vectors of the training images.

Thresholding: Set a threshold value to determine if the distance indicates a valid match. If the distance is below this threshold, the unknown image is considered a match; if not, it may be classified as unknown.

As we compare the threshold value, we can determine wheather or not the image is a match or not, as mentioned just now, this is called thresholding. So essentially, face recognition in PCA, comes down to this very important step as thresholding determines wheather or not any selected image is a match or not

STEP 7 : DETERMINING BEST N PRINCIPAL COMPONENT

- Analyze the explained variance from the principal components and use methods like the elbow method / cross-validation to select the optimal number of components that balance information retention and computational efficiency.



Explained Variance:
Variance for each principal component (eigenvalue) to assess how much information is retained.

Elbow Method:
Plot cumulative explained variance against the number of components and look for an "elbow" point, indicating a good balance between dimensionality reduction and information retention.

Cross-Validation:
Optionally, use cross-validation to evaluate different values of n on a validation set to maximize recognition accuracy.

After thresholding, we come down to selecting the best principal component for said image, this is to ensure that next time when a similar image is shown, the AI can already detect who or what is in this image.

RESULT

Finally, after all the given process, the AI can now differentiate between human faces and non human faces, BASED on its main features after it has been processed mathematically

THANK'S FOR YOUR TIME